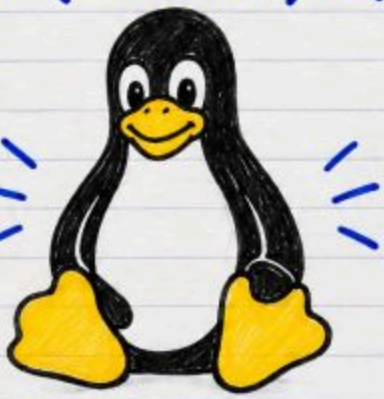
LINUX

COMPLETE NOTES

(Linux)

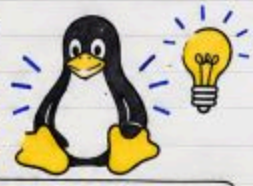


BEGINNER TO ADVANCED

Rakesh Kumar

1. INTRODUCTION TO LINUX

"Linux is a free and open-source operating system based on the Unix kernel. It is secure, stable, and highly customizable."



HISTORY

- Created by Linus Torvalds in 1991.
- Inspired by Unix and developed as a hobby project.
- Released under the GNU General Public License (GPL).
- Today, Linux powers servers, desktops, cloud, mobile devices and more!

FEATURES

- ✓ Open Source – Free to use and modify.
- ✓ Multi-user – Multiple users can work simultaneously.
- ✓ Multi-tasking – Handles multiple tasks efficiently.
- ✓ Secure – Strong permissions and user management.
- ✓ Stable – Runs for long time without reboot.
- ✓ Customizable – Can be adapted to any need.

WHY LINUX ?

- ✓ It's free and open-source.
- ✓ Highly secure and reliable.
- ✓ Preferred choice for servers & cloud.
- ✓ Supports a wide range of hardware.
- ✓ Strong community support.
- ✓ Ideal for developers, DevOps and QA.



LINUX vs WINDOWS

BASIS	LINUX	WINDOWS
License	Open Source (Free)	Proprietary (Paid)
Cost	Free	Paid
Security	More secure, less vulnerable	More vulnerable to malware
Customization	Highly customizable	Limited customization
Performance	Lightweight, faster	Resource intensive
User Interface	Varies (Multiple Desktop Environments)	Graphical, user-friendly
Use Case	Servers, DevOps, Cloud, Development	Gaming, Office, Desktop use

REMEMBER:

- ★ Linux is not just an OS, it's an ecosystem.
- ★ Most of the internet runs on Linux!

Rakesh Kumar

2. LINUX ARCHITECTURE



Linux follows a layered architecture where each layer plays a specific and important role.

USER SPACE
Applications, Programs, User Commands

↕

SHELL
Command Line Interpreter (bash, zsh, etc.)

↕

KERNEL
Core of the OS (Memory, Process, Device, Files, System Calls)

↕

FILE SYSTEM
Manages files, directories, permissions & storage

↕

HARDWARE
CPU, Memory, Disks, Devices, Network, etc.

LAYERS EXPLAINED

1 USER SPACE
- This layer contains all user applications, programs and commands.
- Examples: text editors, browsers, compilers, etc.

2 SHELL
- Acts as an interface between the user and the kernel.
- It takes commands from the user and passes them to the kernel.
- Examples: bash, zsh, sh, etc.

3 KERNEL
- The heart of Linux. It manages system resources, handles system calls and acts as a bridge between software and hardware.
- Examples: process management, memory management, device drivers, etc.

4 FILE SYSTEM
- Organizes and manages data on storage devices.
- It handles files, directories, permissions and disk space.
- Examples: ext4, xfs, btrfs, etc.

5 HARDWARE
- Physical components of the system like CPU, RAM, Disk, Network Cards, I/O Devices, etc.
- Kernel communicates with hardware through device drivers.

HOW IT WORKS ?

User → Shell → Kernel → File System → Hardware

1. User gives a command.
2. Shell interprets it.
3. Kernel processes it.
4. File system reads/writes data.
5. Hardware executes the task.

REMEMBER

- Everything in Linux is a file.
- Kernel is always in control.
- Without kernel, system cannot function.
- Shell is optional, but very powerful.

QUICK COMMANDS

```
uname -a          # System information
cal /proc/version # Kernel details
whoami            # Current user
ls /              # List root directory
ps aux            # Running processes
df -h             # Disk usage
free -m           # Memory usage
```

EXAMPLE: CHECK KERNEL VERSION

```
uname -r          # show kernel version
```

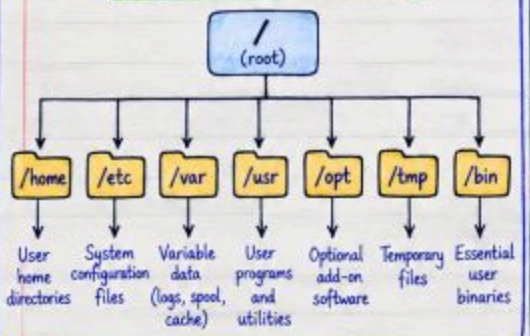
Output: 6.5.0-25-generic

Rakesh Kumar

3. LINUX FILE SYSTEM

Everything in Linux is a file.
Files are organized in a hierarchical tree structure starting from the root (/).

FILE SYSTEM HIERARCHY



EXAMPLE STRUCTURE

```
/
├── home
│   └── rakesh
├── etc
├── var
│   ├── log
│   └── cache
├── usr
│   ├── bin
│   └── local
├── opt
├── tmp
└── bin
```

TIP

- ✓ Root (/) is the base of the entire system.
- ✓ Linux is case-sensitive.
- ✓ Every file or directory inherits permissions from its parent.
- ✓ Use "/" in paths (not \\ like Windows).

IMPORTANT DIRECTORIES

/ (root)
The top of the directory tree. All files and directories start from here.

/home
Contains home directories for users. e.g. /home/rakesh

/etc
Stores system-wide configuration files. Important for system settings.

/var
Holds variable data like logs, spool files, cache, databases, mail, etc.

/usr
Contains most user programs, libraries, documentation and utilities.

/opt
Used to install optional third-party software.

/tmp
Stores temporary files. Files may be deleted on reboot.

/bin
Contains essential command binaries needed for system operation.

QUICK COMMANDS

```
ls /          # List root directory
ls /etc       # List /etc directory
cd /home      # Go to /home
tree /        # Show directory tree (if installed)
```

REMEMBER

- ★ Root (/) is the starting point of everything.
- ★ Keep system directories safe. Do not modify files unless you know what you are doing.
- ★ Use meaningful names for files & directories.

Rakesh Kumar

4. NAVIGATION COMMANDS

Navigation commands help us move around the Linux file system and explore directories.



1 pwd

Print Working Directory

- Shows the current directory you are in.
- Displays the full path.

Example:

```
$ pwd
/home/rakesh/projects
```

2 ls

List files and directories

- Lists files and directories in the current directory.
- Shows the content of a directory.

Example:

```
$ ls
file1.txt  docs  scripts  test
```

3 cd

Change Directory

- Change your current directory to another.
- Use .. to go back to parent directory.

Example:

```
$ cd /home/rakesh/docs
$ pwd
/home/rakesh/docs
```

4 tree

Tree View

- Displays directories in a tree like structure.
- Useful to understand folder structure.

Example:

```
$ tree
.
├── docs
│   ├── file1.txt
│   └── file2.txt
└── scripts
    └── run.sh
```

5 clear

Clear Screen

- Clears the terminal screen for a fresh start.
- Useful when the terminal looks cluttered.

Example:

```
$ clear
# Screen will be cleared
```

6 history

Command History

- Shows previously executed commands.
- Helps to reuse old commands.
- Useful in troubleshooting.

Example:

```
$ history
1  ls
2  cd /home/rakesh
3  pwd
4  tree
5  clear
```

REAL EXAMPLE (PRACTICAL USAGE)

```
1 pwd        # see current location   → /home/rakesh
2 ls         # list files & folders   → docs  scripts  test  notes.txt
3 cd docs    # go to docs folder      → -
4 pwd        # confirm new location   → /home/rakesh/docs
5 tree       # show tree structure    →
                                      .
                                      ├── file1.txt
                                      └── file2.txt
6 cd ..      # go back to parent      → /home/rakesh
7 history    # view command history   → 1 ls | 2 cd docs | 3 pwd | 4 tree | 5 clear
```

REMEMBER

- ⭐ Use Tab key for auto-completion.
- ⭐ Use cd .. to go one level up.
- ⭐ Use cd ~ to go to home directory.

Rakesh Kumar

5. FILE & DIRECTORY COMMANDS

These commands help us to create, manage, copy, move and view files & directories in Linux.

1 mkdir
Make Directory

- Creates a new directory.
- -p creates parent directories if not exist.

Example:
```
$ mkdir myfolder
$ mkdir -p a/b/c
```

2 touch
Create File

- Creates an empty file or updates timestamp.
- Use to create multiple files.

Example:
```
$ touch file.txt
$ touch a.txt b.txt
$ touch {1..3}.txt # multiple files
```

3 cp
Copy Files or Directories

- Copies files or directories from one place to another.
- -r for recursive copy (directory).

Example:
```
$ cp file.txt backup.txt
$ cp -r myfolder newfolder
$ cp -r dir1/ dir2/
```

4 mv
Move or Rename

- Moves files/directories or renames them.
- Used to move a file to another location.

Example:
```
$ mv file.txt docs/
$ mv oldname.txt newname.txt
$ mv myfolder newfolder
```

5 rm
Remove Files

- Deletes files.
- Use -r to delete directories.
- Use -f to force delete (without confirmation).

Example:
```
$ rm file.txt
$ rm a.txt b.txt
$ rm -r myfolder
$ rm -rf dir1/ # force delete
```

6 rmdir
Remove Directory

- Deletes empty directories only.
- Used when directory is empty.
- For non-empty directory, use rm -r.

Example:
```
$ rmdir emptydir
$ rmdir -p a/b/c
$ rm -r nonemptydir # if not empty
```

7 cat
Concatenate & View

- Displays the content of a file on terminal.
- Can also view multiple files.

Example:
```
$ cat file.txt
$ cat file1.txt file2.txt
$ cat -n file.txt # with line no.
```

8 nano
Nano Editor

- Simple and easy text editor in terminal.
- Useful for editing configuration files.

Example:
```
$ nano file.txt
# Ctrl + O -> Save
# Ctrl + X -> Exit
```

9 vi
Vi Editor

- Powerful text editor (Advanced).
- Used in most Linux systems.
- Different modes: insert, command, visual.

Example:
```
$ vi file.txt
  # i   -> Insert mode
  # Esc -> Command mode
  # :wq -> Save & Exit
  # :q! -> Quit without save
```

COMMON OPTIONS

-r	Recursive (for directories)	-n	No overwrite
-f	Force (no confirmation)	-a	Archive (copy with attributes)
-i	Interactive (ask before delete)	--help	Show help
-v	Verbose (show details)	-l	Long listing format
-p	Create parent directories	-h	Human readable size

REMEMBER

- Use cp -r to copy directories.
- Use rm -r to delete directories.
- Use rm -rf carefully (permanent delete).
- Use tab key for auto-completion.
- Use man <command> to get help.
- Be careful while using vi and rm commands.

Rakesh Kumar

7. SEARCHING & TEXT PROCESSING



These commands help us to search for files and text, and process data efficiently in Linux.



#	COMMAND	DESCRIPTION	EXAMPLE	WHAT IT DOES
1	find	Search for files and directories.	$ find /home -name "file.txt"	• Finds file.txt in /home.
2	locate	Find files using pre-built database.	$ locate file.txt	• Fast file search using index.
3	grep	Search for a pattern in files.	$ grep "error" app.log	• Finds lines containing "error".
4	awk	Pattern scanning and processing language.	$ awk '{print $1}' data.txt	• Prints first column from data.txt.
5	sed	Stream editor for filtering & transforming.	$ sed 's/old/new/g' file.txt	• Replaces old with new in file.txt.
6	sort	Sort lines of text files.	$ sort names.txt	• Sorts names in alphabetical order. A↓Z
7	uniq	Remove duplicate lines.	$ uniq list.txt	• Shows only unique lines.
8	wc	Count lines, words and characters.	$ wc file.txt	Counts lines, words and characters.
9	head	Display first few lines of a file.	$ head -n 10 file.txt	↑ Shows first 10 lines.
10	tail	Display last few lines of a file.	$ tail -n 10 file.txt	↓ Shows last 10 lines.

SAMPLE WORKFLOW

1. Find all log files
   $ find /var/log -name "*.log"
   ↓
2. Search for "error"
   $ grep "error" app.log
   ↓
3. Sort and remove duplicates
   $ grep "error" app.log | sort | uniq
   ↓
14. Count total lines
   $ wc -l app.log

EXAMPLE : error.log

```
2024-05-25  ERROR  Failed to start
2024-05-25  INFO   Service running
2024-05-25  ERROR  Connection lost
2024-05-25  WARN   Low memory
2024-05-25  ERROR  Timeout occurred
```

Search ERROR lines:

```
$ grep "ERROR" error.log
2024-05-25  ERROR  Failed to start
2024-05-25  ERROR  Connection lost
2024-05-25  ERROR  Timeout occurred
```

REMEMBER

- grep is case-sensitive.
- locate is faster but may be outdated.
- Use | (pipe) to combine commands.
- Always read man page (man command) for options.
- Use quotes (" ") for patterns with spaces.
- These commands are very useful for log analysis and troubleshooting.

Rakesh Kumar

8. PROCESS MANAGEMENT



These commands help us view, control and manage processes running in a Linux system.



#	COMMAND	DESCRIPTION	EXAMPLE	WHAT IT DOES
1	ps	Report snapshot of current processes.	$ ps aux	Shows currently running processes.
2	top	Real-time view of processes.	$ top	Displays dynamic process information.
3	htop	Interactive process viewer (better than top).	$ htop	User-friendly real-time process monitor.
4	jobs	List background and stopped jobs.	$ jobs	Shows jobs started from current shell.
5	bg	Resume a stopped job in background.	$ bg %1	Continues job in background.
6	fg	Bring a background job to foreground.	$ fg %1	Brings job to current terminal.
7	kill	Terminate a process using PID.	$ kill 1234	Sends TERM signal (default).
8	kill -9	Force kill a process (SIGKILL).	$ kill -9 1234	Forcefully terminates unresponsive process.
9	nice	Run a process with modified priority.	$ nice -n 10 command	Adjusts process priority (lower value = higher priority).

EXAMPLES

```
1 $ ps aux          # View all running processes
2 $ top             # Real-time monitoring
3 $ htop            # Better process viewer
4 $ jobs            # List background jobs
5 $ bg %1           # Resume job in background
6 $ fg %1           # Bring job to foreground
7 $ kill 1234       # Kill process with PID
8 $ kill -9 1234    # Force kill process
9 $ nice -n 10 cmd  # Run with low priority
```

JOB STATES

- Running (R) – Running in foreground.
- Sleeping (S) – Running in background / waiting.
- Stopped (T) – Stopped (Use Ctrl + Z).
- Zombie (Z) – Process completed but not removed.

Example:

```
$ jobs
[1]+ Running    sleep 100 &
[2]- Stopped    vim file.txt
```

REMEMBER

- Use ps aux for detailed process information.
- top is useful but htop is easier to use.
- Use Ctrl + Z to stop a process and bg to run in background.
- kill sends default TERM signal (kind request).
- kill -9 should be used only when process does not stop.
- nice command starts process with lower priority (higher number = lower priority).

Rakesh Kumar

9. DISK & MEMORY COMMANDS

These commands help us monitor disk usage, memory usage and system performance.



#	COMMAND	DESCRIPTION	EXAMPLE	WHAT IT DOES
1	df	Disk free space report.	$ df -h	Shows disk space usage of file systems.
2	du	Estimate file and directory space usage.	$ du -sh /home/user	Shows size of files or directories.
3	free	Display memory usage.	$ free -h	Shows total, used and free memory.
4	uptime	Show how long system has been running.	$ uptime	Shows system uptime and load average.
5	vmstat	Report virtual memory statistics.	$ vmstat 1 5	Shows processes, memory, CPU and I/O stats.
6	iostat	I/O statistics for CPU and devices.	$ iostat -xz 1 5	Shows CPU and device I/O performance.
7	mount	Mount a file system.	$ mount /dev/sdb1 /mnt	Attaches a filesystem to a directory.
8	umount	Unmount a file system.	$ umount /mnt	Detaches a filesystem from a directory.
9	lsblk	List block devices.	$ lsblk	Shows information about block devices.
10	df -i	Show inode usage.	$ df -i	Shows inode usage of file systems.

EXAMPLES

```
$ df -h              # disk usage in human readable
$ du -sh /var/log    # size of /var/log
$ free -h            # memory usage
$ uptime             # system uptime
$ vmstat 1 5         # memory & CPU stats
$ iostat -xz 1 5     # I/O statistics
$ mount /dev/sdb1 /mnt # mount device
$ umount /mnt        # unmount device
$ lsblk              # list block devices
$ df -i              # inode usage
```

SAMPLE OUTPUTS

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   20G   28G  42% /
/dev/sdb1       100G   60G   40G  60% /data
```

```
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7.7G        2.1G        2.3G        0.3G        3.3G        4.8G
Swap:          2.0G          0B        2.0G
```

```
$ uptime
 14:35:21 up 3 days,  2:45,  2 users,
 load average:  0.08,  0.12,  0.15
```

REMEMBER

- Use `df -h` to check disk space regularly.
- Use `free -h` to monitor memory usage.
- High load average means system is busy.
- Use `vmstat` and `iostat` for performance analysis.
- Always unmount before removing devices.
- Use `lsblk` to identify devices before mounting.
- Use `df -i` to check inode usage.

TIP Combine commands with pipes for powerful reports!

`$ df -h | grep sda` | `$ du -hs * | sort -h` | `$ free -h | grep Mem`

Rakesh Kumar

10. NETWORKING COMMANDS

These commands help us to troubleshoot, test connectivity and manage network connections in Linux.

#	COMMAND	DESCRIPTION	EXAMPLE	WHAT IT DOES
1	ping	Test connectivity between hosts.	$ ping google.com	Checks if host is reachable and measures latency.
2	curl	Transfer data to/from a server.	$ curl -I https://example.com	Fetches data or HTTP headers from URL.
3	wget	Download files from the web.	$ wget https://file.com/file.zip	Downloads files non-interactively.
4	ssh	Securely connect to a remote server.	$ ssh user@server.com	Opens secure remote shell session.
5	scp	Securely copy files between hosts.	$ scp file.txt user@host:/path	Copies files over SSH securely.
6	netstat	Show network connections, routing and interfaces.	$ netstat -tulnp	Displays active connections and listening ports.
7	ss	Modern replacement for netstat.	$ ss -tulnp	Faster way to show sockets and connections.
8	nslookup	Query DNS to find IP or domain information.	$ nslookup example.com	Looks up DNS records (A, AAAA, CNAME, MX etc).
9	dig	Advanced DNS lookup tool.	$ dig example.com	Detailed DNS query and troubleshooting.
10	traceroute	Trace the path packets take to a host.	$ traceroute google.com	Shows each hop to the destination.

EXAMPLES

```
1 $ ping -c 4 google.com
  # send 4 packets
2 $ curl -I https://example.com
  # fetch headers
3 $ wget https://file.com/file.zip
  # download file
4 $ ssh -p 22 user@192.168.1.10
  # connect on port 22
5 $ scp report.pdf user@server:/home/user/
  # copy file to remote server
6 $ netstat -tulnp
  # show listening ports
7 $ ss -tulnp | grep 80
  # check process on port 80
8 $ dig short example.com
  # get only IP address
```

COMMON PORTS

PORT	SERVICE
20 / 21	FTP
22	SSH
23	Telnet
25	SMTP
53	DNS
80	HTTP
443	HTTPS
3306	MySQL
6379	Redis
8080	Application (Alt)

REMEMBER

- ping checks connectivity, not speed.
- curl is great for APIs and headers.
- Use ssh and scp for secure file transfer.
- ss is faster and more powerful than netstat.
- dig gives more detailed DNS information than nslookup.
- traceroute helps find network issues (hops where it fails).
- Use -p option to specify port in ssh and scp.
- Always check firewall and security group if connection fails.

TIP Combine commands with pipes for powerful network troubleshooting:
$ ping google.com | grep ttl ; $ ss -tulnp | grep 80 ; $ dig example.com | grep ANSWER

Rakesh Kumar

11. PACKAGE MANAGEMENT

These commands help us install, update, remove and manage software packages in Linux.

#	COMMAND	DESCRIPTION	EXAMPLE	WHAT IT DOES
1	apt	Advanced Package Tool (Debian / Ubuntu).	$ apt search nginx	• Package manager for Debian based systems.
2	yum	Yellowdog Updater Modified (RHEL/CentOS).	$ yum list installed	• Package manager for RHEL/CentOS (v7 and below).
3	dnf	Dandified YUM (Fedora/RHEL 8+).	$ dnf repolist	• Next-generation package manager (Fedora, RHEL 8+).
4	rpm	Red Hat Package Manager.	$ rpm -qa | grep httpd	• Manages RPM packages (install, query, verify).
5	dpkg	Debian Package Manager.	$ dpkg -l | grep vim	• Manages DEB packages on Debian based systems.
6	install	Install a package.	$ apt install nginx	• Installs a software package.
7	update	Update package lists or upgrade packages.	$ yum update	• Updates package lists or installed packages.
8	remove	Remove a package.	$ apt remove nginx	• Removes a package from the system.

PACKAGE MANAGERS

- Debian / Ubuntu — apt, dpkg
- RHEL / CentOS 7 — yum, rpm
- Fedora / RHEL 8+ — dnf, rpm

COMMON OPERATIONS EXAMPLES

APT (Debian/Ubuntu)

```
$ sudo apt update      # update list
$ sudo apt upgrade     # upgrade all
$ sudo apt install nginx
  # install package
$ sudo apt remove nginx
  # remove package
$ apt search docker
```

YUM/DNF (RHEL/CentOS/Fedora)

```
$ sudo yum repolist          # show repos
$ sudo yum list installed    # list packages
$ sudo yum install nginx     # install package
$ sudo yum update            # update packages
$ sudo yum remove nginx      # remove package
$ dnf search docker          # search package
```

USEFUL OPTIONS

```
-y          # automatic yes
-q          # quiet mode
--search    # search packages
list        # list installed packages
update      # update repository
upgrade     # upgrade all packages
remove      # remove a package
--reinstall # reinstall a package
```

RPM vs DEB

RPM (Red Hat)

- Used in RHEL, CentOS, Fedora.
- File extension: .rpm
- Install: rpm -ivh file.rpm
- Query: rpm -qa
- Remove: rpm -e package

DEB (Debian)

- Used in Debian, Ubuntu.
- File extension: .deb
- Install: dpkg -i file.deb
- Query: dpkg -l
- Remove: dpkg -r package

REMEMBER

- Use sudo with package commands.
- Run update before install/upgrade.
- apt = Debian / Ubuntu
- yum = RHEL / CentOS 7
- dnf = RHEL 8+ / Fedora
- rpm & dpkg work on package files directly.
- Always check package name before install/remove.

Rakesh Kumar

12. SHELL SCRIPTING BASICS

Shell scripting lets us automate tasks by writing commands in a file and executing them.

1 VARIABLES

- Used to store data.
- No spaces around = sign.
- Access value with $ before name.

x = 10
name = QA

Example:

```
#!/bin/bash
name="Rakesh"
count=5
echo "Hello $name, count is $count"
```

2 IF CONDITION

- Used for decision making.
- Common operators: -eq, -ne, -gt, -lt, -ge, -le

YES
NO

Example:

```
#!/bin/bash
read -p "Enter a number: " num
if [ $num -gt 10 ]; then
    echo "Number is greater than 10"
else
    echo "Number is 10 or less"
fi
```

3 LOOPS

- for loop: iterate over items.
- while loop: repeat while condition is true.

For loop Example:

```
#!/bin/bash
for i in 1 2 3 4 5; do
  echo "Number: $i"
done
```

While loop Example:

```
#!/bin/bash
count=1
while [ $count -le 5 ]; do
  echo "Count: $count"
  count=$((count+1))
done
```

4 FUNCTIONS

- Group commands into reusable blocks.
- Call by function name.

Example:

```
#!/bin/bash
greet() {
    echo "Hello $1! Welcome to Shell Scripting."
}
greet "Rakesh"    # function call
```

5 ARGUMENTS

- $0 → script name.
- $1, $2, ... → positional arguments.
- $# → total number of arguments.
- $@ → all arguments.
- $* → all arguments as a single string.

Example:

```
#!/bin/bash
echo "Script: $0"
echo "First Arg: $1"
echo "Second Arg: $2"
echo "Total Args: $#"
echo "All Args: $@"
```

6 SIMPLE SCRIPT EXAMPLE

Take user input, show date, and list files in current directory.

myscript.sh

```
1  #!/bin/bash
2  echo "Enter your name: "
3  read name
4  echo "Hello $name!"
5  echo "Today's date and time: "
6  date
7  echo "Files in current directory: "
8  ls -l
9  echo "Script execution completed."
10 exit 0
```

Run the Script:

```
chmod +x myscript.sh
./myscript.sh
```

Output (Example):

```
Enter your name: Rakesh
Hello Rakesh!
Today's date and time:
Fri Oct 30 21:45:00 IST 2026
Files in current directory:
total 8
-rw-r--r-- 1 user user 0 file1.txt
-rw-r--r-- 1 user user 0 file2.txt
Script execution completed.
```

REMEMBER

- Give execute permission: chmod +x script.sh
- Run script: ./script.sh
- Use # for comments.
- Use meaningful variable and function names.

TIP

- Start small, test often.
- Break big tasks into small scripts.
- Use variables, loops and functions to automate real work.

Rakesh Kumar

13. LINUX IN REAL PROJECTS

Linux is the backbone of modern IT infrastructure.
It powers tools, servers, pipelines and platforms.

WHERE IS LINUX USED ?

1 SOFTWARE DEVELOPMENT



- Preferred OS for developers.
- Supports Git, Vim, GCC, Make, Node.js, Python, Java etc.
- Easy package management and environment setup.
- Lightweight, stable and highly customizable.

2 CLOUD & DEVOPS



- Most Linux servers run on AWS, Azure and GCP.
- Used in CI/CD, monitoring, logging and automation.
- Core for Docker, Kubernetes, Terraform, Ansible.
- Used for building and managing cloud infrastructure.

3 CYBER SECURITY



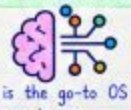
- Preferred by security professionals.
- Tools like Kali Linux, Parrot OS.
- Used for penetration testing, vulnerability assessment and digital forensics.
- Tools: Nmap, Wireshark, Metasploit, Burp Suite.

4 DATA ENGINEERING



- Linux handles big data workloads.
- Runs Hadoop, Spark, Kafka, Airflow, Hive etc.
- Stable, secure and high performance for large datasets.
- Widely used in data pipelines and analytics platforms.

5 AI / MACHINE LEARNING



- Linux is the go-to OS for AI/ML engineers.
- Supports Python, TensorFlow, PyTorch, CUDA etc.
- Works seamlessly on GPUs and high performance clusters.
- Used for model training, experimentation and deployment.

6 WEB HOSTING



- Most web servers run on Linux.
- Hosts Nginx, Apache, MySQL, PHP, Node.js applications.
- Reliable, lightweight, secure and cost-effective.
- Used for websites, web apps and APIs.

REAL PROJECT WORKFLOW



DEVELOPER	GIT	LINUX SERVER	DOCKER	JENKINS	KUBERNETES	CLOUD
Writes code and tests locally on Linux.	Code is committed and pushed to repository.	Code is pulled on Linux server for build and execution.	Application is containerized using Docker.	Builds, tests and creates artifacts.	Deploys and manages containers at scale.	Application runs on cloud (AWS / Azure / GCP).

WHY LINUX ?



- ✓ Open Source and Free.
- ✓ Secure and Stable.
- ✓ Powerful Command Line.
- ✓ Automates Repetitive Tasks.
- ✓ Runs Everywhere (On-prem, Cloud, Edge).
- ✓ Huge Community Support.

TIP



Learn Linux well, and you can work in any tech role with confidence!

REMEMBER



- ★ Linux is not just an OS, it's the foundation of modern IT.
- ★ From coding to deploying, Linux is everywhere!
- ★ It helps in building reliable, scalable and secure systems.
- ★ Master Linux = Career Booster.

> "Behind every great project, there is a powerful Linux system running silently."

Rakesh Kumar

15. CHEAT SHEET & ROADMAP

TOP 50 LINUX COMMANDS

#	COMMAND	PURPOSE
1	pwd	Print working directory
2	ls	List files and directories
3	cd	Change directory
4	mkdir	Create a directory
5	rmdir	Remove an empty directory
6	touch	Create an empty file
7	cp	Copy files or directories
8	mv	Move or rename files
9	rm	Remove files or directories
10	cat	Display file contents
11	less	View file content (scrollable)
12	head	Show first lines of a file
13	tail	Show last lines of a file
14	nano	Edit files in terminal
15	vi	Edit files (advanced)
16	chmod	Change file permissions
17	chown	Change file owner
18	ps	Show running processes
19	top	Monitor system processes
20	kill	Terminate a process
21	df	Disk space usage
22	du	Directory space usage
23	free	Show memory usage
24	uname	System information
25	hostname	Show or set hostname

#	COMMAND	PURPOSE
26	whoami	Show current user
27	id	Show user and group IDs
28	grep	Search text in files
29	find	Search files and directories
30	locate	Find files by name (updatedb)
31	which	Locate command path
32	echo	Display text
33	history	Show command history
34	man	Show manual of a command
35	whatis	Brief description of command
36	alias	Create command shortcut
37	tar	Archive files
38	gzip	Compress files
39	gunzip	Decompress .gz files
40	zip	Zip files
41	unzip	Unzip files
42	scp	Secure copy (remote files)
43	ssh	Secure shell login
44	ping	Check connectivity
45	netstat	Network connections
46	ss	Socket statistics
47	curl	Transfer data from URL
48	wget	Download files from web
49	crontab	Schedule tasks (cron jobs)
50	clear	Clear the terminal screen

LINUX LEARNING ROADMAP

1. **Basics**
   - Linux introduction
   - File system
   - Navigation
   - Basic commands
2. **Commands**
   - File operations
   - Text processing
   - Permissions
   - Users & groups
3. **System Admin**
   - Processes
   - Services
   - Logs & monitoring
   - Scheduling (cron)
4. **Networking**
   - IP, DNS, SSH
   - Networking tools
   - Firewall basics
   - Troubleshooting
5. **DevOps Tools**
   - Git
   - Docker
   - Jenkins
   - CI/CD basics
6. **Orchestration**
   - Kubernetes
   - YAML basics
   - Deploy apps
   - Scaling & HA
7. **Cloud**
   - AWS / Azure / GCP
   - Linux on Cloud
   - Automation
   - Real projects

REMEMBER

- Practice commands daily.
- Use man <command> for help.
- Learn by doing real tasks.
- Small steps, big progress!

TIPS

- ✓ Start with basic commands.
- ✓ Be consistent and practice.
- ✓ Make your own cheat sheet.
- ✓ Try real projects and scenarios.
- ✓ Use a Linux VM or cloud server.

BEST PRACTICES

- ✓ Use documentation.
- ✓ Automate repetitive tasks.
- ✓ Keep system updated.
- ✓ Take regular backups.
- ✓ Follow security best practices.
- ✓ Practice troubleshooting.

> “Linux is not just an OS, it’s a skill that powers your DevOps journey!”

Rakesh Kumar